Check-in 3 — core watermark benchmark

CMPE 261 Sec 01, Fall 2026. Group 6. Kanaka Sarat Siripurapu (019132776), Latha Boralingaiah (018301361), Yashashav Devalapalli Kamalraj (017856371).

Repository: https://github.com/yashashav-dk/cmpe261-watermark-forgery Team Drive: https://drive.google.com/drive/folders/1wkXOxPjfI648JPod1Fi2m9ItfrPvWVIT Deadline: November 3, 2026, 11:59 PM. Deliverable: PDF draft report.

Draft status: implementation prepared; full Check-in 3 measurements are pending. Historical numbers below come from the corrected Check-in 2 report. They are not Check-in 3 results. GPU compatibility, complete measurements, actual team contributions, and access checks must be resolved before submission.

1. Updated abstract and report outline

Semantic watermarks associate diffusion outputs with a secret signal recovered by inversion. We study false attribution: modifying an innocent real photograph so that a provider's detector accepts it as watermarked. We benchmark TR, RI, SFW, GS, WIND, and PRC on pinned Stable Diffusion 2.1-base. Earlier experiments established forgery on all six schemes at preliminary scale. Check-in 3 replaces those measurements with independent clean and real-photo baselines, calibrated score tests, fixed-budget imprint forgery, and four removal attacks. Native and calibrated decisions remain separate, including a separately labeled PRC score-only test. Rates receive Wilson intervals; continuous metrics receive 10,000-resample bootstrap intervals. The implementation preserves optimization state and measured images through interruption. Full benchmark results remain pending; this draft makes no claim that the intended experiment has completed.

The final report contains: introduction and threat model; related work; datasets and reproducibility; methods and algorithms; experimental protocol; results and uncertainty; limitations and discussion; team contributions and check-ins; references. This draft supplies the methods and protocol and labels measurement-dependent sections as pending.

2. Literature survey and threat model

Tree-Ring embeds a frequency-domain pattern in starting noise; RingID extends this family toward identifying multiple keys [1,2]. Gaussian Shading encodes secret bits while maintaining a Gaussian noise distribution [3]. PRC uses a pseudorandom error-correcting code, with a native decoder decision distinct from a scalar score test [4]. WIND uses noise-based identification [5]. SFW is the sixth selected toolkit scheme; its native distance score follows the same orientation convention as TR and RI. We use the common MarkDiffusion interface at a fixed source revision [9].

The imprint attack of Mueller et al. [16] is the direct methodological basis. It optimizes an innocent cover's VAE latent until inversion approaches the inverted latent of a watermarked reference, without optimizing against the provider's detector. The detector is used only for measurement. Stopping after a positive detection would introduce detector-dependent stopping; the current protocol always reaches its fixed cap. A later latent-noise attack [17] and removal benchmarks such as WAVES [10] motivate comparing forgery and removal under an explicit common protocol, without claiming this project is the first forgery study.

The attacker has one watermarked reference and the same publicly available model as the target (condition C1). Access to the native detector is available to the evaluator. The attack does not receive that verdict as an optimization objective or stopping condition. Model transfer, reprompting, additional schemes, and extended attacks are outside this milestone.

3. Dataset, configuration, and reproducibility

Retain the historical 50 prompts and 50 COCO val2017 identifiers as exact prefixes. Extend to 200 prompts and 100 photograph identifiers from the same input pools with seed 261. Select the first ten photographs as forgery covers, preserving the original five. The selector rejects a different existing selection. Snapshot preprocessed photograph pixels and hash all inputs before starting measurement. The 200-prompt and 100-photo selections have been generated from the source datasets; data/checkin3-selection.json records pool and selection hashes.

Target and attacker: Manojb/stable-diffusion-2-1-base, revision 0094d483a120f3f33dafbd187ea4aa60d10de75c. MarkDiffusion revision: 9d81656d1a5f9e5194fc2f727bb795ef29e53809. Images are RGB, 512 by 512. Generation and inversion use 50 steps; generation guidance is 7.5. Both pipelines use float32. Imprint uses Adam at learning rate 0.01. The target retains the existing DPMSolverMultistep scheduler; the attacker uses DDIM inversion.

The requested GPU stack is Python 3.12, Torch 2.10.0 with CUDA 12.8, and Diffusers 0.40.0. The setup pins MarkDiffusion and supplies qrcode, galois, and ldpc below 2. Compatibility is a gate, not an assumed result. Capture the complete installed environment after successful import checks and pip check. Preserve the original run's environment on resume.

A root manifest binds code hashes, environment, model/toolkit revisions, seeds, input identities and hashes, counts, attack settings, precision, and budgets. A per-scheme manifest binds the complete saved toolkit state, including keys and configuration. The state serializer rebinds shared pipeline components instead of storing duplicate model weights. Actual toolkit round trips require the GPU preflight; synthetic round trips alone are insufficient evidence.

4. Approach and algorithms

F1 generates independent watermarked and unwatermarked images, saves each artifact, and evaluates those persisted pixels. Baseline images are never silently reused from historical runs. F3 consumes the saved F1 watermarked images, with hashes linking each transformed artifact to its source.

Imprint computes the reference's inverted noise once, optimizes the cover latent toward it, and decodes the current optimized latent every ten steps. Optimization continues through step 150 even if the detector later reverses a positive verdict. The checkpoint contains the latent, Adam state, target inverted noise, Python / NumPy / Torch / CUDA random states, elapsed synchronized optimization work, per-step loss records, evaluated image, and its measurement.

Publication order is checkpoint, measured image, then versioned log. The checkpoint commits the measurement before it enters the log. Reopening replays publication idempotently and continues from the saved optimizer state; it never recomputes an old trajectory while retaining its measurements. A reference provenance record commits its pixels, prompt, seed, and identity before optimization. Changed references, measured images, configuration, keys, or input hashes reject resume.

A single-writer lock guards the full output directory. Log keys include protocol, scheme, stage, attack identity, image identifier, kind, and step. Different removal attacks cannot suppress each other's measurements. Historical CSV readers and the smoke-run interface remain available. A fresh numbered pilot directory prevents accidental overwriting of successful preflight evidence.

5. Experimental protocol and statistics

F1: per scheme, 100 watermarked generations, 200 unwatermarked generations, and 100 real photographs. Include a scheme in attack comparisons only when its observed native clean TPR is at least 90% and generated-image FPR is at most 5%. Report exclusions with their native baseline results. An initial detection miss never removes an image from the removal denominator.

Calibrate using only the 200 unwatermarked scores. Orient distance scores by negation (TR, RI, SFW); GS, WIND, and the separate PRC score test use increasing scores. For sorted scores s1 through s200, use threshold s198 and the strict test score > threshold. At most two calibration images pass; ties can make this fewer. Call this an empirical calibration operating point with 0.5-percentage-point resolution. Its in-sample rate is not independent proof of a population FPR. Keep PRC native decoder verdicts and its score-only calibrated test separate.

F2: the same ten covers for every included scheme. Run all 150 steps and evaluate every ten. Primary success rates use the actual saved images at steps 50, 100, and 150, with PSNR, SSIM, and LPIPS at those budgets. Never substitute an earlier positive detection. Save trajectories as secondary diagnostics.

F3: apply JPEG quality 95, Gaussian blur radius 2, centered crop-and-scale at ratio 0.8, and pinned SD 2.1-base diffusion purification at strength 0.3 with an empty prompt. Seed stochastic transforms per image. Evaluate saved transformed artifacts. The pinned image_editor.py source was audited: crop resize uses Pillow’s default BICUBIC filter; purification uses the toolkit’s unguided DiffusionPurification editor. The GPU pilot remains required before claiming end-to-end reproduction.

Use Wilson 95% intervals for binary rates and percentile bootstrap intervals with 10,000 item resamples and seed 261 for mean continuous metrics. Report each metric's finite observation count. Bootstrap over photographs, never over optimization steps. Incomplete cells use observed denominators and are visibly provisional; they cannot substitute for required full sample sizes. No ranking claim is justified merely by small numerical differences or overlapping plots.

6. Results and interpretation

Check-in 3 full F1: pending, 0 of 6 schemes verified at 100 / 200 / 100. Check-in 3 full F2: pending, 0 of 60 cover trajectories verified through 150 steps. Check-in 3 full F3: pending, 0 of 2,400 transformed images verified (before any native inclusion exclusions). Actual GPU cost and complete confidence intervals are pending. The executable report reads only the new versioned log and lists missing baseline, eligibility, and attack work explicitly.

Historical comparison only, corrected Check-in 2: each scheme used 50 watermarked, 50 generated negatives, and 50 photographs. GS, PRC, WIND, RI, and SFW were forged on 5 of 5 covers; TR on 4 of 5. Those give Wilson 95% intervals of about 57–100% and 38–96%, respectively. Clean native TPR was 50/50 except RI and SFW at 49/50; TR generated FPR was 2/50 and other generated/real FPR cells were 0/50.

Historical median first-detection steps were GS 10, PRC 20, WIND 30, RI 70, SFW 90, and TR 70. These are stopping measurements, not fixed-budget success rates, and cannot establish a current scheme ranking. Historical GPU timing was 4.9 seconds per imprint step on an RTX PRO 6000 and 8.4 seconds on an A100. These do not establish the cost of the new environment or a T4 run.

Replace the pending paragraph with the completeness-checked summary and figures only after full execution. Present native and calibrated F2 next to native and calibrated F3, with full denominators and uncertainty. Retain baseline failures and initial misses in the reported accounting.

7. Verification, limitations, contributions, and schedule

The local full suite passed 95 tests, including the cached tiny diffusion model. The final validation record is results/checkin3/software-validation.json. Local regression coverage includes independent baseline counts, strict threshold ties, PRC rule separation, attack-specific keys, saved artifacts, native inclusion, fixed-budget reversal, single-writer enforcement, immutable reference provenance, and interruptions before and after checkpoint/log publication. An independent code review identified preflight evidence overwrite, hidden attack requirements, CUDA timer synchronization, and incomplete reference provenance; each received a regression test and a fix. Actual toolkit serialization and GPU execution remain unverified; do not interpret CPU tests as evidence of GPU correctness.

The connected pilot session exposed a T4 with 15 GB GPU memory, Python 3.13, and initially no Drive mount. Drive was then mounted. The isolated Python 3.12 GPU stack and full six-scheme pilot have not completed. No lower precision, reduced sample count, weaker attack, or shorter optimization budget is authorized as an automatic workaround. Preserve partial progress and report outstanding work.

Use approximately nine meaningful implementation and report commits, allocated three per repository member as requested by the team. Commit attribution follows the existing repository identities. This allocation is not a statement of each member's personally completed tasks. Commit timestamps are intentionally spaced 2.5 hours apart for presentation and do not measure elapsed work. Before submission, each member must confirm their actual implementation, experiment, analysis, and writing contributions. The nine task commits rotate in this order: (1) Kanaka Sarat — versioned measurements and calibration; (2) Latha — optimizer checkpoint recovery; (3) Yashashav — reproducibility setup and key state; (4) Kanaka Sarat — fixed-budget runner; (5) Latha — exact-budget reporting; (6) Yashashav — GPU preflight and recovery verification; (7) Kanaka Sarat — Colab driver; (8) Latha — report draft; (9) Yashashav — frozen dataset selections.

Previous responsibilities were metrics/statistics/provenance (Kanaka Sarat), attack implementation/transferability (Latha), and generation/environment (Yashashav); these remain responsibilities rather than verified new contributions.

The corrected Check-in 2 report states that the instructor and TA were invited to GitHub and Drive before Check-in 1. Current access has not been reverified. Missing invitations need the account owner's help. Do not claim a new access check has passed based solely on that historical statement.

October 24–25: environment, persistence, and GPU pilot. October 26–28: full F1/F2. October 29–30: full F3 and completeness audit. October 31–November 1: analysis, figures, report, and targeted reproduction. November 2: PDF review and preparation. November 3 is contingency. GitHub publication and Canvas submission remain separate actions. Full queue planning estimate: 16–20 RTX PRO 6000 hours, subject to measured pilot validation and additional setup overhead.

8. References and reproducible entry points

References below retain the source list used in the corrected Check-in 2 report; the final submission should retain the full reviewed bibliography in checkin-2.md.

[1] Wen et al. Tree-Ring Watermarks. NeurIPS 2023. arXiv:2305.20030. [2] Ci et al. RingID. ECCV 2024. arXiv:2404.14055. [3] Yang et al. Gaussian Shading. CVPR 2024. arXiv:2404.04956. [4] Gunn, Zhao, Song. An Undetectable Watermark for Generative Image Models. 2024. arXiv:2410.07369. [5] Arabi et al. Hidden in the Noise: Two-Stage Robust Watermarking for Images. 2024. arXiv:2412.04653. [9] Pan et al. MarkDiffusion. 2025. arXiv:2509.10569. [10] An et al. WAVES: Benchmarking the Robustness of Image Watermarks. ICML 2024. [12] Song, Meng, Ermon. Denoising Diffusion Implicit Models. ICLR 2021. arXiv:2010.02502. [16] Mueller et al. Black-Box Forgery Attacks on Semantic Watermarks for Diffusion Models. CVPR 2025. arXiv:2412.03283. [17] Forging and Removing Latent-Noise Diffusion Watermarks Using a Single Image. 2025. arXiv:2504.20111.

Notebook: notebooks/03_checkin3_colab.ipynb. Tests: PYTHONPATH=src python -m pytest -q -m 'not slow'. Pilot: python -m wmforge.preflight --out PILOT_DIR. Full run: python -m wmforge.checkin3 --out RUN_DIR --preflight PILOT_DIR/preflight.json. Summary: python -m wmforge.checkin3_report --run RUN_DIR. PDF: python scripts/checkin3_draft.py --out docs/Group6_CMPE261_CheckIn_3_draft.pdf.

The full run must use the compatibility-checked CUDA environment. Source, input selection, key state, checkpoints, artifacts, calibration, and summary together form the reproduction record. Never publish private watermark keys by accident.